

Sans documents excepté une copie double resto-verso avec notes manuscrites

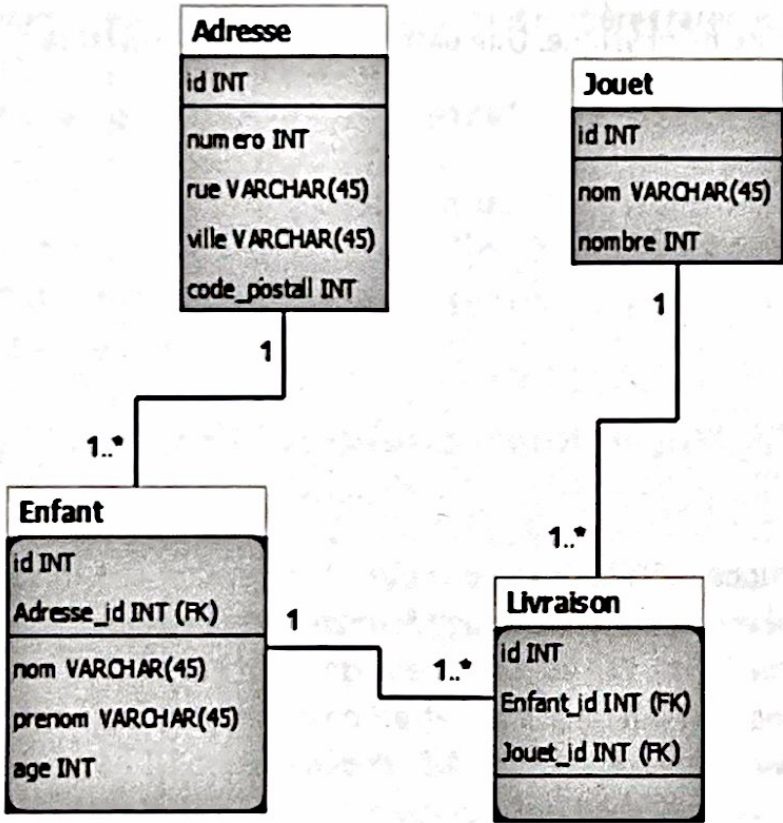
Les paramètres des fonctions sont parfois volontairement omis, à vous de les définir. La clarté de vos réponses sera prise en compte.
N'oubliez pas de commenter vos programmes

Vous aurez besoin de 3 copies : une par partie

Partie 1 : Base de données (7 points) -> Copie n°1

Livraisons du Père Noël

Pour l’organisation et le suivi de ses livraisons de jouets aux enfants le Père Noël dispose de la base de données composée des tables décrites dans le schéma relationnel ci-dessous.
L’attribut **nombre** de la relation **Jouet** représente le stock disponible du jouet décrit par son id et son nom.



1. Écrire la requête SQL permettant de rechercher tous les noms et prénoms des enfants habitant à « Compiègne ».
2. Écrire la requête SQL qui retourne les *id* des jouets de l’enfant « Romy Trannoy ». Dans le cas où il y aurait plusieurs enfants de mêmes nom et prénom, la requête retournera les *id* pour un seul enfant.
3. Écrire la fonction Python `livraison_jouets()` ayant comme paramètres le curseur associé à la base de données et une ville. Cette fonction retournera la liste des *id* des jouets à livrer pour la ville passée en paramètre.
4. Écrire la fonction Python `decremente_stock()` ayant comme paramètres le curseur associé à la base de données, ainsi qu’une liste contenant les *id* des jouets livrés (un *id* par jouet) pour lesquels on doit décrémenter le stock. Au cas où il n’y aurait plus de stock, aucune action ne sera effectuée.
5. Écrire le programme principal qui ouvre la base de données `noel_2025.sqlite`, demande le nom de la ville que le Père Noël doit livrer et met à jour le stock de jouets en fonction de cette livraison. Le programme traitera les exceptions pouvant être déclenchées lors de ce programme.
6. Dans quelle table doit-on ajouter un attribut `livré` qui indique qu’un jouet a bien été livré à un enfant ?

Partie 2 : Objets, fichiers, interfaces graphiques (6 points) -> Copie n°2

Gestionnaire d'Alertes Vigicrues

Dans le cadre de la modernisation du réseau Vigicrues du bassin de la Seine, vous êtes chargé de développer le module central de collecte d'alertes. Ce système doit être capable de recevoir des données provenant de multiples capteurs hydrologiques et d'enregistrer des alertes dans un journal centralisé. Le gestionnaire d'alertes Vigicrues a une architecture basée objet qui repose essentiellement sur 2 classes : **JournalAlerte** et **Capteur**.

N.B. Les fonctions `date()` et `heure()` sont supposées connues. Elles retournent selon le cas une chaîne de caractères comprenant la date ou l'heure courante.

1. Écrire en Python la classe **JournalAlerte** qui gère l'écriture dans un fichier texte.

À la création de l'objet, il reçoit et stocke comme attribut, le chemin d'un fichier (ex : `log_alertes.txt`). Ce fichier doit être créé avec une ligne d'en-tête comprenant un titre et la date (ex : `--- JOURNAL DES CRUES 14/01/2026---`).

Sa méthode `ecrire_alerte(self, message)` ouvre le fichier en mode ajout et y inscrit le message reçu suivi d'un saut de ligne. 

2. Soit la classe **Capteur** qui représente une sonde physique. Une partie du code est la suivante :

```
class Capteur:
```

```
    def __init__(self, ville, journal) -> None:
        if type(ville) != str or not isinstance(journal, JournalAlerte):
            raise TypeError("Erreur dans le type des attributs de Capteur")
        self._ville = ville
        self._journal = journal
```

```
    def mesure(self) -> float:
        # méthode supposée connue, retourne la mesure du niveau d'eau en mètres
```

Compléter la classe **Capteur** avec la méthode `effectuer_releve()` qui réalise une mesure de niveau et retourne un message. Si le niveau dépasse 8 mètres, elle prépare un message formaté incluant le nom de la ville, le niveau et l'heure actuelle (ex : `ALERTE [12:00:00] Paris : niveau 8.5 m`). Elle utilise la méthode `ecrire_alerte()` de **JournalAlerte** pour enregistrer l'alerte dans le fichier. Si le niveau est inférieur à 8 mètres, on n'écrit rien dans le fichier et le message retourné indiquera qu'il n'y a rien à signaler pour la ville concernée (ex : `R.A.S. Rouen (6.25 m)`).

3. Pour rendre le système plus convivial, on voudrait concevoir une petite fenêtre d'affichage (*Figures ci-contre*). L'interface affichera un bouton « *relever les capteurs* » qui, lorsqu'il est cliqué, déclenche l'affichage d'un cycle de mesures sur tous les capteurs.

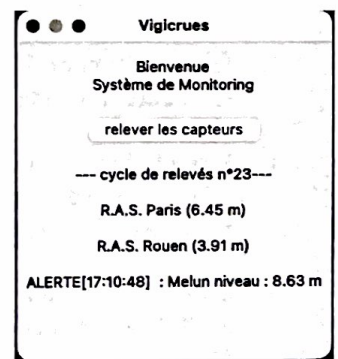
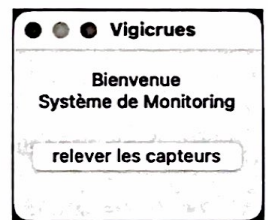
Soit la classe **VigicruesApp** dont une partie du code est donnée ci-dessous.

Écrire la méthode `declencher_mesures(self)` qui affiche le numéro de cycle de relevés, parcourt la liste des capteurs, effectue les relevés pour chacun d'entre eux et affiche les messages. Le message de chaque relevé doit être ajouté au fur et à mesure dans le label prévu pour l'affichage en utilisant la variable de contrôle `self._releve` (voir figure ci-contre, affichage après 23 clics sur le bouton).

```
class VigicruesApp (tk.Tk):
```

```
    def __init__(self, liste_capteurs):
        super().__init__()
        self.title("Vigicrues")
        self._capteurs = liste_capteurs
        self._releve = tk.StringVar() # pour l'affichage des relevés, "" par défaut
        self._num_releve = tk.IntVar() # numéro du relevé, 0 par défaut
        # Création des widgets
        tk.Label(self, text="Bienvenue\nSystème de Monitoring").pack(pady=10)
        tk.Button(self, text="relever les capteurs", command=self.declencher_mesures).pack(pady=5)
        tk.Label(self, textvariable=self._releve).pack(pady=10) # zone affichage
```

```
    def declencher_mesures(self) -> None: # À FAIRE
```



4. Écrire le programme principal qui instancie un objet **JournalAlerte** pour recueillir les messages de crues dans le journal nommé `log_alertes.txt` et crée une liste contenant 3 objets **Capteur** situés respectivement à Paris, Rouen et Melun et qui partagent tous le même journal. Le programme lance ensuite l'interface graphique permettant à l'utilisateur de déclencher les cycles de mesures et visualiser les résultats dans la zone d'affichage.

Partie 3 : ndarray, courbes, fichier csv (7 points) -> Copie n°3

Dans cette partie, on exploitera lorsque c'est possible, les possibilités offertes par numpy pour les *ndarray* (méthodes et fonctions).

Exercice 3.1

Soit un fichier au format csv géré par le ministère de l'éducation nationale comprenant pour tous les bacheliers de France : le nom, le numéro de l'académie¹, et les notes de quelques matières. Un extrait de ce fichier est donné ci-contre.

Extrait de fichier *bachelier.csv*

```
Nom;Académie;Français;Math;Sport
ABOUR;1;12.25;7.75;16.90
ACER;28;15.85;13.75;15.83
ACOSTANT;7;6.65;4.75;18
AMAURY;9;9.85;7;15.25
ANGEL;2;14;12;17.45
ANTOIN;3;9.85;6.5;16.5
ARBELAID;22;13.05;12;18.16
BALBANY;18;16.95;11.25;16.24
BAIFAR;16;4.4;2.5;16.22
BALLIGANO;9;5.85;11;16.4
...
```

1. Écrire une fonction `charge_fichier()` qui lit le fichier au format csv dont le nom est passé en paramètre, et retourne un *ndarray* 2D contenant les données du fichier anonymisées (sans le nom et sans les entêtes de colonnes). La fonction retournera aussi la liste des noms de colonnes de ces données. Ces deux éléments seront appelés `data` et `entete` dans la suite de l'exercice.

Par exemple avec le fichier *bachelier.csv* :

- Le *ndarray* `data` contient en 1ère colonne le numéro d'académie et les colonnes suivantes contiennent les notes dans les matières retenues : Français, Math et Sport, ceci pour tous les bacheliers.
 - la liste `entete` sera égale à `["Académie", "Français", "Math", "Sport"]`.
2. Écrire la fonction `affiche_histo_moy(data)` qui affiche l'histogramme des moyennes de l'ensemble des bacheliers contenus dans `data` comme le montre la figure 3.1 (les données sont fictives).
 3. Écrire la fonction `meilleure_matiere(data, entete)` qui retourne le nom de la matière qui a la meilleure moyenne.
 4. Écrire la fonction `courbe_moy_academie(data)` qui affiche sous forme de courbe la moyenne générale par académie. Un exemple est montré dans la figure 3.2 (30 académies).
 5. On demande dans la question 1 de construire un *ndarray* 2D. Aurait-il été possible de construire un *ndarray* 3D dans lequel on aurait : par ligne un bachelier, par colonne les différentes matières et par plan l'académie ? Justifier en quelques mots.

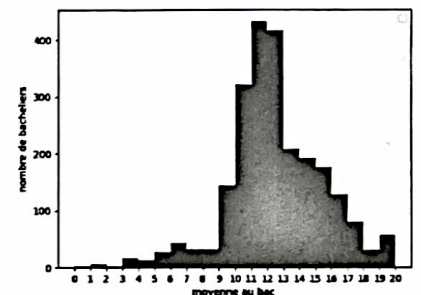


Fig. 3.1 Histogramme des moyennes

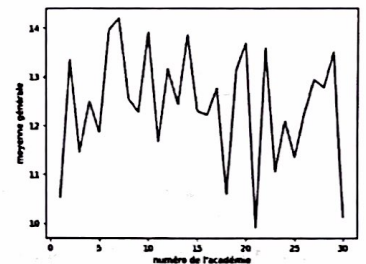


Fig. 3.2 Moyenne générale par académie

Exercice 3.2

6. Soit une image couleur chargée dans un *ndarray* `im_couleur`. Cette image couleur est codée sur 4 canaux (RVBA), R pour le canal rouge, V pour le canal vert, B pour le canal bleu et A pour la transparence. Écrire un script python qui crée et affiche une image en niveau de gris (sans transparence) en prenant le minimum de chaque canal (rouge, vert ou bleu).

¹ Une académie est une région administrative pour l'enseignement primaire et secondaire.